

Number Factor Problem

<https://replit.com/@ElshadKarimov/NumberFactorDP#NumberFactor.java>

Solution to House Robber Top Down Approach

```
public class HouseRobber {

    public int maxMoneyTopDown(int[]
HouseNetWorth) {
        int dp[] = new
int[HouseNetWorth.length];
        return maxMoneyTopDown(dp,
HouseNetWorth, 0);
    }

    private int maxMoneyTopDown(int[] dp,
int[] HouseNetWorth, int currentIndex) {
        if (currentIndex >=
HouseNetWorth.length)
            return 0;

        if (dp[currentIndex] == 0) {
            int stealCurrent =
HouseNetWorth[currentIndex] +
maxMoneyTopDown(dp, HouseNetWorth,
currentIndex + 2);
            int skipCurrent =
maxMoneyTopDown(dp, HouseNetWorth,
currentIndex + 1);
            dp[currentIndex] =
Math.max(stealCurrent, skipCurrent);
        }
        return dp[currentIndex];
    }
}
```

```

    public int maxMoneyBottomUp(int[] wealth) {
        int dp[] = new int[wealth.length + 2];
        // '+1' to handle the zero house
        dp[wealth.length] = 0; // if there are
        no houses, the thief can't steal anything
        for (int i = wealth.length - 1; i >=
0; i--) {
            dp[i] = Math.max(wealth[i] + dp[i
+ 2], dp[i + 1]);
        }
        return dp[0];
    }
}

```

Solution to House Robber Bottom Up Approach

```

public class HouseRobber {

    public int maxMoneyTopDown(int[]
HouseNetWorth) {
        int dp[] = new
int[HouseNetWorth.length];
        return maxMoneyTopDown(dp,
HouseNetWorth, 0);
    }

    private int maxMoneyTopDown(int[] dp,
int[] HouseNetWorth, int currentIndex) {

```

```

        if (currentIndex >=
HouseNetWorth.length)
            return 0;

        if (dp[currentIndex] == 0) {
            int stealCurrent =
HouseNetWorth[currentIndex] +
maxMoneyTopDown(dp, HouseNetWorth,
currentIndex + 2);
            int skipCurrent =
maxMoneyTopDown(dp, HouseNetWorth,
currentIndex + 1);
            dp[currentIndex] =
Math.max(stealCurrent, skipCurrent);
        }
        return dp[currentIndex];
    }

    public int maxMoneyBottomUp(int[] wealth) {
        int dp[] = new int[wealth.length + 2];
        // '+1' to handle the zero house
        dp[wealth.length] = 0; // if there are
no houses, the thief can't steal anything
        for (int i = wealth.length - 1; i >=
0; i--) {
            dp[i] = Math.max(wealth[i] + dp[i
+ 2], dp[i + 1]);
        }
        return dp[0];
    }
}

```

Solution to Convert One String to Another using Top Down Approach

```
public class ConvertOneStringToAnother {

    public int findMinOperationsTD(String
s1, String s2) {
        Integer[][] dp = new
Integer[s1.length()+1][s2.length()+1];
        return findMinOperationsTD(dp, s1, s2,
0, 0);
    } //end of method

    private int
findMinOperationsTD(Integer[][] dp, String s1,
String s2, int i1, int i2) {
        if(dp[i1][i2] == null) {
            if(i1 == s1.length()) // if we have
reached the end of s1, then insert all the
remaining characters of s2
                dp[i1][i2] = s2.length() - i2;

            else if(i2 == s2.length()) // if we
have reached the end of s2, then delete all
the remaining characters of s1
                dp[i1][i2] = s1.length() - i1;

            else if(s1.charAt(i1) ==
s2.charAt(i2)) // If the strings have a
matching character, recursively match for the
remaining lengths.
                dp[i1][i2] =
findMinOperationsTD(dp, s1, s2, i1+1, i2+1);

            else {
```

```

        int c1 = findMinOperationsTD(dp,
s1, s2, i1+1, i2); //delete
        int c2 = findMinOperationsTD(dp,
s1, s2, i1, i2+1); //insert
        int c3 = findMinOperationsTD(dp,
s1, s2, i1+1, i2+1); //replace
        dp[i1][i2] = 1 + Math.min(c1,
Math.min(c2, c3));
    }
    } //end of if-else
    return dp[i1][i2];
} //end of method

```

```

    public int findMinOperationsBU(String s1,
String s2) {
        int[][] dp = new int[s1.length() +
1][s2.length() + 1];

        for (int i1 = 0; i1 <= s1.length(); i1++)
// if we have reached the end of s1, then
insert all the remaining characters of s2
        dp[i1][0] = i1;

        for (int i2 = 0; i2 <= s2.length(); i2++)
// if we have reached the end of s2, then
delete all the remaining characters of s1
        dp[0][i2] = i2;

        for (int i1 = 1; i1 <= s1.length(); i1++)
{
            for (int i2 = 1; i2 <= s2.length();
i2++) { // If the strings have a matching
character, recursively match for the remaining
lengths.

```

```

        if (s1.charAt(i1 - 1) == s2.charAt(i2
- 1))
            dp[i1][i2] = dp[i1 - 1][i2 - 1];
        else
            dp[i1][i2] = 1 + Math.min(dp[i1 -
1][i2], // delete
Math.min(dp[i1][i2 -
1], // insert
dp[i1 - 1][i2 - 1]));
// replace
    }
}
return dp[s1.length()][s2.length()];
}

} //end of class

```

Solution to Convert One String to Another using Bottom Up Approach

```

public class ConvertOneStringToAnother {

    public int findMinOperationsTD(String
s1, String s2) {
        Integer[][] dp = new
Integer[s1.length()+1][s2.length()+1];
        return findMinOperationsTD(dp, s1, s2,
0, 0);
    } //end of method
}

```

```

        private int
findMinOperationsTD(Integer[][] dp, String s1,
String s2, int i1, int i2) {
    if(dp[i1][i2] == null) {
        if(i1 == s1.length()) // if we have
reached the end of s1, then insert all the
remaining characters of s2
            dp[i1][i2] = s2.length() - i2;

        else if(i2 == s2.length()) // if we
have reached the end of s2, then delete all
the remaining characters of s1
            dp[i1][i2] = s1.length() - i1;

        else if(s1.charAt(i1) ==
s2.charAt(i2)) // If the strings have a
matching character, recursively match for the
remaining lengths.
            dp[i1][i2] =
findMinOperationsTD(dp, s1, s2, i1+1, i2+1);

        else {
            int c1 = findMinOperationsTD(dp,
s1, s2, i1+1, i2); //delete
            int c2 = findMinOperationsTD(dp,
s1, s2, i1, i2+1); //insert
            int c3 = findMinOperationsTD(dp,
s1, s2, i1+1, i2+1); //replace
            dp[i1][i2] = 1 + Math.min(c1,
Math.min(c2, c3));
        }
    } //end of if-else
    return dp[i1][i2];
} //end of method

```

```

    public int findMinOperationsBU(String s1,
String s2) {
        int[][] dp = new int[s1.length() +
1][s2.length() + 1];

        for (int i1 = 0; i1 <= s1.length(); i1++)
// if we have reached the end of s1, then
insert all the remaining characters of s2
            dp[i1][0] = i1;

        for (int i2 = 0; i2 <= s2.length(); i2++)
// if we have reached the end of s2, then
delete all the remaining characters of s1
            dp[0][i2] = i2;

        for (int i1 = 1; i1 <= s1.length(); i1++)
        {
            for (int i2 = 1; i2 <= s2.length();
i2++) { // If the strings have a matching
character, recursively match for the remaining
lengths.
                if (s1.charAt(i1 - 1) == s2.charAt(i2
- 1))
                    dp[i1][i2] = dp[i1 - 1][i2 - 1];
                else
                    dp[i1][i2] = 1 + Math.min(dp[i1 -
1][i2], // delete
                                                Math.min(dp[i1][i2 -
1], // insert
                                                dp[i1 - 1][i2 - 1]));
// replace
            }
        }
        return dp[s1.length()][s2.length()];
    }

```



```
}
```

```
//end of class
```